

Atividade 04 – Herança

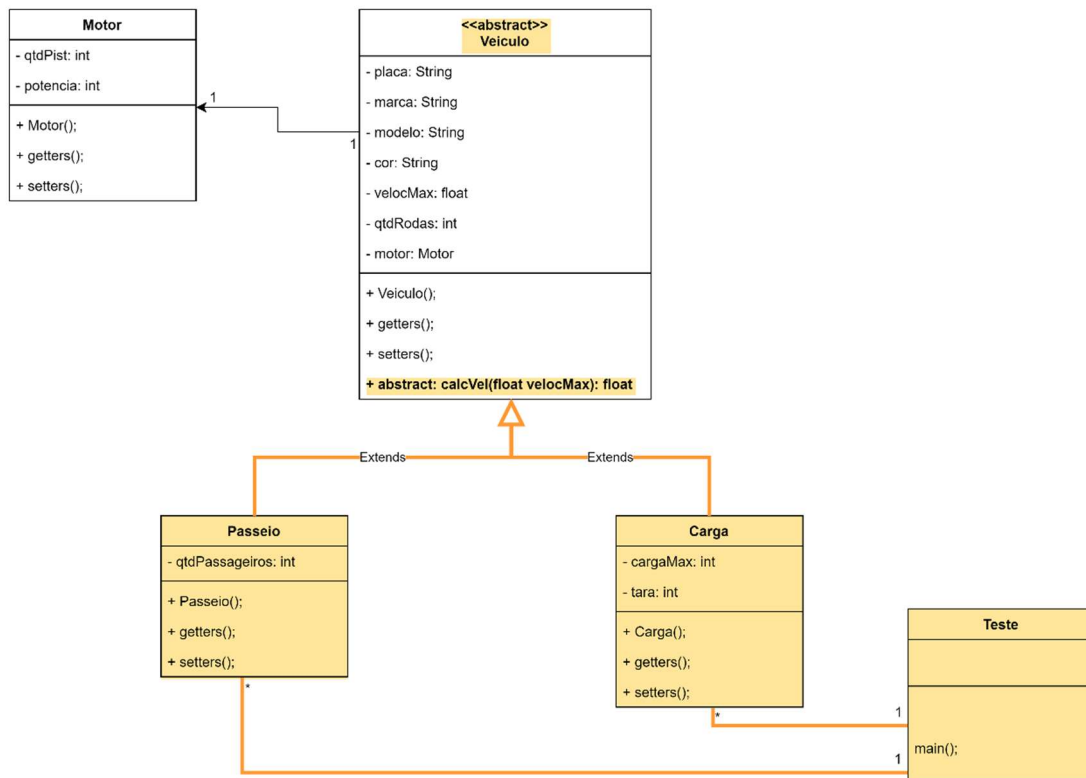
→ Este exercício trata-se de uma evolução da Atividade 01.

Embora a Atividade 4 trate do conceito e aplicação do mecanismo de Herança, ela também pode nos ajudar compreender a realidade das empresas e instituições que buscam intensificar o uso de padrões de projetos desenvolvidos, de maneira *ad hoc*, por elas mesmas, com intuito de padronizar a construção de seus softwares.

Por exemplo, ao definir-se como "final" um método "set", indicando que não poderá ser sobrescrito, garantimos a perpetuação de uma regra de negócio restritiva sobre as características possíveis de um objeto (um atributo deste), como quando não queremos que um atributo receba um valor fora de uma escala prevista.

1) OBSERVE O SEGUINTE DIAGRAMA DE CLASSES

Obs.: As alterações e novos elementos referentes à atividade encontram-se marcadas e amarelo e laranja (relacionamentos) no diagrama.



2) UTILIZE O CÓDIGO DESENVOLVIDO NA ATIVIDADE 01 E DESENVOLVA OS NOVOS ELEMENTOS APRESENTADOS NO DIGRAMA ACIMA. ABAIXO, SEGUIE A LISTA DE REQUISITOS A SEREM SEGUIDOS:

- a. A “entrada” da velocidade (atributo *velocMax*) sempre será dada em km/h porém, a exibição destes dados ocorrerá na classe Teste e da seguinte forma:
 - i. A velocidade do veículo de **passeio** deverá ser calculada em m/h;

1 kilometer/hour = 1000 meter/hour

- ii. A velocidade do veículo de **carga** deverá ser calculada em cm/h;

1 kilometer/hour = 100000 centimeter/hour

Use o método `calcVel(float velocMax)`, da classe-mãe, para fazer este cálculo.

Atenção:

- O método `calcVel(float velocMax)` **NÃO** deve alterar o valor do atributo `velocMax`, **apenas convertê-lo** e retornar o valor convertido para que seja exibido na tela por meio da classe `Teste`;

- b. Os métodos construtores default deverão iniciar com 0 (zeros) atributos que sejam de tipos numerais (int, double, float, etc.) e com espaço em branco os que forem de tipo literais (char, String e etc.).
- c. Garanta que nunca ocorra:
- i. As classes **Passeio** e **Carga** **jamaís** deverão ser estendidas (herdadas);
 - ii. Nenhum método “set” poderá ser sobrescrito;
- d. A classe “Teste” deve ser construída de forma a testar todas as funcionalidades do programa (entrada, saída e cálculos), propiciando assim “trocas de mensagens” entre os objetos das classes `Teste` ↔ `Passeio` e `Teste` ↔ `Carga`. Por meio dela deverá ser possível instanciar 5 veículos de cada tipo (Passeio/Carga).

3) O QUE SERÁ AVALIADO

- a. Construção das classes, com os atributos e métodos conforme descritos no diagrama de classe do item 01.
- b. Relacionamento de herança entre as classes.
- c. Cada uma das solicitações presentes no item 2.
- d. Implementação da relação entre as classes `Teste` -> `Passeio`, `Teste` -> `Carga`, conforme solicitado no item 2.d.
- e. Uso do encapsulamento.

Importante!

- Atenha-se aos nomes dos elementos (classes, atributos e métodos) conforme apresentados no diagrama.
- Novos métodos poderão ser criados, caso julgue necessário.
- Os itens avaliados são os solicitados no enunciado. Elementos extras **NÃO** renderão pontos a mais.
- O não cumprimento do que foi solicitado acarretará no decréscimo da nota de acordo com a gravidade da falta.
- A justificativa para qualquer desconto será colocada, pelo avaliador, no campo de feedback de cada Atividade.